Чеклист инструментов и навыков к практике 2

На практике вы построите в PostgreSQL две связанные витрины по заказам и платежам. Перед занятием проверьте инструменты и базовый SQL.

Инструменты

Если в первой практике все успешно запустилось и подключилось - пропускаем. Новых инструментов не будет.

- ☐ **Docker Desktop запускается.** В терминале выполните `docker version` и `docker compose version`: обе команды должны завершиться без ошибки. Если программы нет, установите её по инструкции Docker для macOS или Windows.
  - ☐ Если хочется подробнее разобраться в Docker, **пройдите официальный quickstart Docker:** Build and share a containerized application. После него вы должны различать образ и контейнер, понимать назначение опубликованного порта и уметь запускать `compose.yaml` командами `docker compose up -d`, `docker compose ps`, `docker compose down`.
- ☐ **DBeaver Community открывается и умеет подключаться к PostgreSQL.** Скачать: DBeaver Community. При создании соединения выберите PostgreSQL. Если DBeaver предложит загрузить драйвер, подтвердите.
- ☐ **Python 3 и Jupyter-ноутбук запускаются в VS Code, подключение к PostgreSQL работает.** Нужно успешно выполнить ячейку Проверка подключения к PostgreSQL из Практика_2_старт.ipynb. Для новой установки: Python 3, VS Code.

Проверка учебного PostgreSQL

Из папки с файлами практики выполните:

```
docker compose up -d
docker compose ps
```

В `docker compose ps` у сервиса `postgres` должно появиться состояние `healthy`. Подключение из DBeaver: `localhost:5435`, база `shop`, пользователь `student`, пароль `studentpass`.

Проверьте соединение одним запросом в DBeaver:

```
select status, count(*) as orders_count
from orders
group by status
order by status;
```

В результате должны быть пять статусов; для `paid` — `107` заказов. После проверки можно остановить стенд командой `docker compose down` из той же папки. Данные в томе сохранятся.

SQL: что нужно уметь до практики

Проверьте, что можете прочитать и написать запрос с конструкциями ниже. На практике они будут применяться вместе.

Конструкция	Что нужно уметь
`select ... from ...`, `as`	Выбрать нужные столбцы и дать им понятные псевдонимы
`where`, `=`, `<>`, `>`, `and`, `or`, `in`, `is null`	Отобрать строки по статусу, идентификатору и наличию значения; понимать, что `null` проверяют через `is null`
`order by`, `limit`	Упорядочить результат и вывести первые строки для проверки
`count(*)`, `sum(...)`, `min(...)`, `group by`	Посчитать число и сумму операций и получить одну строку на `order_id`
`having count(*) > 1`	Найти ключи, которым соответствует несколько строк
`join ... on ...`, `left join ... on ...`	Соединить заказы и платежи; понимать, что `left join` сохраняет строку слева при отсутствии совпадения
`create table`, `insert into ... select ...`, `truncate table`	Создать учебную витрину, заполнить её результатом запроса и очистить перед повторной загрузкой

Проверьте себя на запросе выше: объясните, какую роль играют `count(*)`, `group by` и `order by`. Затем составьте запрос, который выводит `order_id` и `total_amount` для заказов со статусом `paid`, отсортированных по `order_id`, и ограничьте вывод пятью строками.

Если базовый SQL пока не получается, пройдите любой вводный курс с упражнениями по `select`, `where`, агрегатам и `group by`, например «Основы работы с базами данных и SQL» Яндекс Практикума .

Что будет применяться из лекции 2

Эти конструкции разбираются на лекции и закрепляются в практике; предварительно достаточно узнавать их в запросе:

Конструкция	Зачем она понадобится
`with ... as (...)`	Отдельно агрегировать платежи до одной строки на заказ перед соединением
`row_number() over (partition by ... order by ...)`	Найти первый подтверждённый заказ каждого клиента
`lag(...) over (...)`	Посмотреть предыдущую покупку в истории клиента
`count(*) over (...)`, `sum(...) over (...)`	Посчитать итоги клиента, сохранив отдельные строки заказов
`coalesce(..., 0)`	Показать ноль операций для заказа без успешного платежа

Главный вопрос перед каждым соединением: **сколько строк соответствует одному заказу или клиенту до и после** `join`**?** Он поможет не завысить сумму заказов при нескольких платежных операциях.